Navigation Algorithm Comparison Report

A* | Dijkstra | BFS | DFS on Real Beijing Road Network

1. Project Overview

This project compares four pathfinding algorithms on a real Beijing road network (Zhongguancun area), using real taxi GPS trajectories from the Microsoft T-Drive dataset.

Road Network: OpenStreetMap via osmnx — 413 nodes, 945 edges

Trajectory Data: Microsoft T-Drive — 6,666 taxi files, 1,319 valid trajectories extracted

Valid Criteria: Start ≠ End, path exists, straight-line distance > 100m

2. Algorithm Description

Algorithm	Strategy	Data Structure	Optimality
A*	f(n) = g(n) + h(n), heuristic = Haversine	Min-heap	Optimal
Dijkstra	f(n) = g(n) + 0, no heuristic	Min-heap	Optimal
BFS	Expand level by level, ignores weights	Queue (FIFO)	Fewest hops only
DFS	Go deep first, backtrack when stuck	Stack (LIFO)	No guarantee

Key relationship: Dijkstra is A* with h(n) = 0. Comparing them directly reveals the value of the heuristic function.

3. Overall Results (1,319 Trajectories)

Algorithm	Avg Path (m)	Avg Nodes	Avg Time (ms)	Quality Ratio
A*	**2,747.7**	**111.9**	0.71	**1.00 (baseline)**
Dijkstra	2,747.7	298.6	0.94	1.00
BFS	2,833.8	233.7	0.19	1.03
DFS	6,244.3	141.5	0.14	3.00

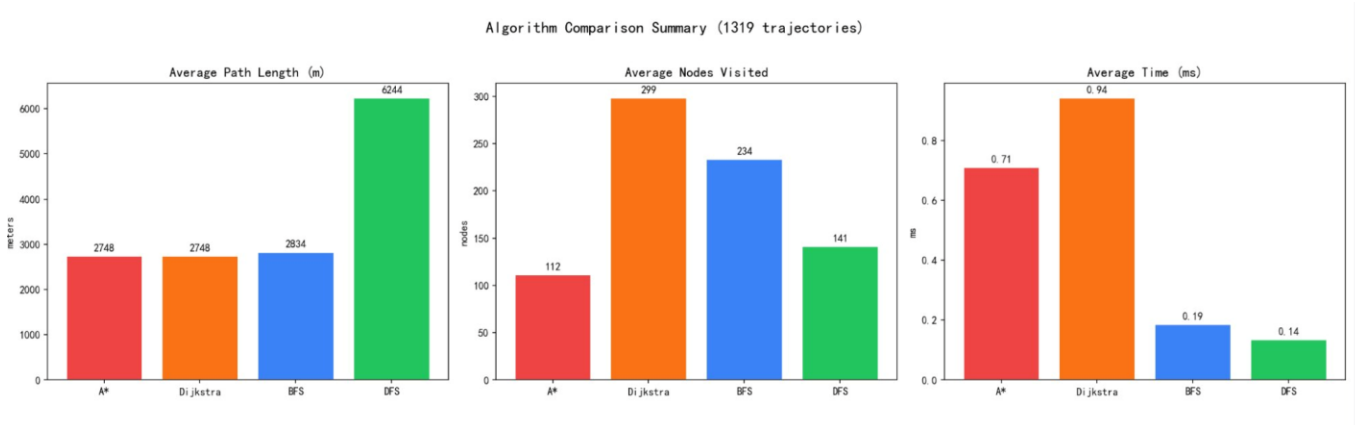

Figure 1: Algorithm Comparison Summary (1,319 trajectories)

Key Findings

A* achieves the shortest path with the fewest nodes visited. It explores only 112 nodes on average to find the optimal route, while Dijkstra needs 299 nodes to find the exact same path. This directly proves the value of the heuristic function — it reduces unnecessary exploration by 62.5%.

Dijkstra finds the same optimal path as A*, but visits 2.7× more nodes. Since Dijkstra has no heuristic guidance, it searches in all directions equally, wasting effort on nodes far from the goal.

BFS finds near-optimal paths (quality ratio 1.03) but visits many nodes. BFS ignores edge weights and optimizes for fewest hops, not shortest distance.

DFS produces the worst paths — on average 3× longer than A*. With a quality ratio of 3.00 and high variance, DFS is fundamentally unsuitable for navigation.

4. Results by Distance Group

Short Distance (< 1 km) — 264 trajectories

Algorithm	Avg Path (m)	Avg Nodes	Avg Time (ms)
A*	1,041.3	21.7	0.14
Dijkstra	1,041.3	74.3	0.23
BFS	1,056.8	52.9	0.04
DFS	4,753.2	147.9	0.14

For short distances, A* needs only ~22 nodes. DFS paths are 4.6× longer than optimal.

Medium Distance (1–3 km) — 1,025 trajectories

Algorithm	Avg Path (m)	Avg Nodes	Avg Time (ms)
A*	3,142.1	129.9	0.83
Dijkstra	3,142.1	352.2	1.12
BFS	3,244.1	276.1	0.22
DFS	6,517.7	138.2	0.13

The majority of trajectories (78%) fall in this range. A* visits 130 nodes vs Dijkstra's 352 — the heuristic saves 63% of exploration effort.

Long Distance (> 3 km) — 30 trajectories

Algorithm	Avg Path (m)	Avg Nodes	Avg Time (ms)
A*	4,290.1	291.7	1.84
Dijkstra	4,290.1	442.8	1.38
BFS	4,455.0	378.5	0.33
DFS	10,021.4	198.3	0.18

For long routes, DFS paths exceed 10 km — 2.3× the optimal distance.

5. Path Quality Analysis

The path quality ratio measures each algorithm's path length divided by A*'s path length. A ratio of 1.0 means the path is optimal.

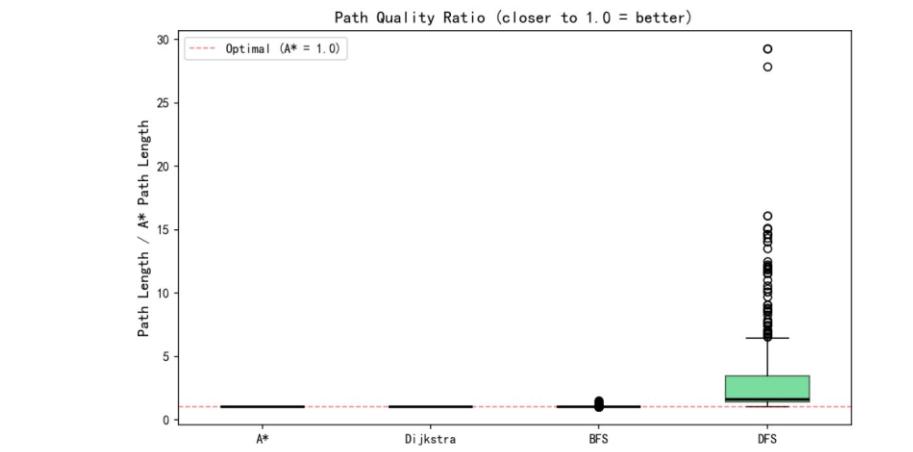

Figure 2: Path Quality Ratio Distribution (closer to 1.0 = better)

The box plot shows that DFS has extreme variance — some paths are up to 29× longer than optimal. A*, Dijkstra, and BFS are tightly clustered around 1.0, demonstrating consistent performance.

6. Path Visualization (Example: 40.txt)

Figure 3: Four Algorithm Paths on Zhongguancun Road Network

A* (red, 3,725m): Shortest, most direct route

Dijkstra (orange, 3,725m): Identical path to A*, confirming both find the true optimum

BFS (blue, 3,764m): Slightly different route, optimizing for fewest intersections

DFS (green, 3,966m): Visibly longer path with unnecessary detours

7. Why A* Has Higher Execution Time

A* shows a higher average time (0.71ms) than BFS (0.19ms) and DFS (0.14ms). This is not a weakness — it is explained by the small network size (413 nodes).

A* performs a Haversine distance calculation for every neighbor it evaluates, plus heap insertion/extraction. BFS and DFS use simple queue/stack operations with near-zero per-node cost. On a network this small, the per-node overhead of A* dominates.

On city-scale networks with hundreds of thousands of nodes, A*'s advantage of visiting far fewer nodes would translate into a significant time advantage. The 62.5% reduction in nodes visited (112 vs 299 compared to Dijkstra) would save far more time than the per-node computation cost.

8. Conclusion

Based on 1,319 real taxi trajectories tested on the Beijing Zhongguancun road network:

1. A* is the best overall algorithm for navigation. It finds the shortest path while visiting the fewest nodes (112 avg), making it the most efficient in terms of search quality.

2. Dijkstra confirms A*'s optimality. Both find identical paths (quality ratio 1.00), but Dijkstra visits 2.7× more nodes, directly proving the heuristic function reduces unnecessary exploration by 62.5%.

3. BFS is acceptable but inefficient. Path quality is close to optimal (ratio 1.03), but it visits more nodes than necessary because it ignores edge weights.

4. DFS is unsuitable for navigation. Paths are on average 3× longer than optimal with high variance (up to 29× in worst cases).

Future Work

Consider comparing A* with more advanced algorithms such as Bidirectional A*, Contraction Hierarchies, or ALT (A* with Landmarks and Triangle inequality) to further explore the trade-off between preprocessing time and query speed.